Dominer la course TORCS : guide stratégique complet pour l'IBM AI Racing League

Les contrôleurs hybrides — architectures modulaires à règles, optimisées par algorithmes évolutionnaires — dominent sans conteste les compétitions TORCS SCR depuis 2009. AUTOPIA (logique floue + algorithme génétique) Springer et Mr. Racer (apprentissage de piste + CMA-ES) ont remporté la quasi-totalité des championnats entre 2009 et 2015, surpassant largement les approches pures d'apprentissage par renforcement (RL) ou les contrôleurs naïfs à règles fixes. Pour l'IBM AI Racing League 2026 — qui impose un client Python snakeoil3 sur le circuit Corkscrew — les gains les plus rapides proviennent de cinq modifications ciblées : passage aux 6 vitesses par RPM, vitesse cible dynamique basée sur les capteurs, contrôle sigmoïde accélération/freinage, direction atténuée en vitesse, et ABS logiciel. Ce rapport détaille l'état de l'art, les résultats connus, les paramètres critiques et les recommandations concrètes.

L'état de l'art des compétitions SCR TORCS depuis 2007

La compétition Simulated Car Racing (SCR) a été lancée lors du **WCCI 2008** par Daniele Loiacono, Pier Luca Lanzi (Politecnico di Milano) et Julian Togelius. ResearchGate togelius Elle a fonctionné de 2008 à environ 2015, à travers les conférences CIG, GECCO, CEC et EVO*. Le format comprenait une phase d'échauffement (100 000 ticks pour l'apprentissage en ligne), une qualification solo (10 000 ticks) et une course multi-voitures (top 8, 5 tours, système de points F1). Slideshare +3 Toutes les équipes utilisaient la voiture standardisée **car1-trb1** arXiv +2 et les pistes étaient inconnues à l'avance — générées procéduralement Slideshare — forçant la généralisation. togelius En 2010, un **bruit gaussien de 10 %** a été ajouté aux capteurs, bouleversant le classement. ResearchGate

L'interface SCR expose Sbgames **19 capteurs de distance** (range finders de -90° à +90°), la vitesse (longitudinale et latérale), le RPM, l'angle par rapport à l'axe de la piste, la position sur la piste (trackPos), les dégâts, et les vitesses de rotation des roues. IEEE Xplore arXiv Les actionneurs sont la direction ([-1, +1]), l'accélérateur ([0, 1]), le frein ([0, 1]), le rapport de vitesse ({-1, 0, 1...6}) et l'embrayage. GitHub +6

Les approches gagnantes : une hiérarchie claire

AUTOPIA (Onieva, Pelta, Alonso, Milanés, Pérez — universités de Madrid et Grenade) reste le contrôleur le plus dominant de l'histoire SCR. Son architecture modulaire comprend six modules — changement de vitesse, direction (logique floue), contrôle de vitesse (logique floue), gestion des pédales (heuristique), évitement d'adversaires (modificateur flou) et apprentissage inter-tours. Springer +2 Les fonctions d'appartenance floues sont optimisées hors ligne par **algorithme génétique** sbgames avec une fitness basée sur la distance parcourue et des pénalités de crash. Sbgames +3 AUTOPIA a dominé les championnats **2010** et **2011** (100 points contre 61,5 pour le deuxième) SlideShare et était le meilleur marqueur de la manche GECCO 2013. Slideshare

Mr. Racer (Jan Quadflieg, Mike Preuss, Günter Rudolph — TU Dortmund) utilise une approche « humaine » avec apprentissage de la géométrie de la piste pendant l'échauffement, planification anticipée de la trajectoire, et optimisation des paramètres par **CMA-ES.** Semantic Scholar +3 Il a remporté le championnat global SCR en **2013** Slideshare Slideshare et figurait systématiquement dans le top 3. Slideshare Sa faiblesse : une sensibilité au bruit des capteurs introduit en 2010. ResearchGate

COBOSTAR (Butz et Lönneker, Université de Würzburg) repose sur des couplages sensori-moteurs paramétrés, IEEE Xplore avec **14 paramètres** optimisés par CMA-ES. IEEE Xplore ResearchGate Il a gagné la manche CEC-2009 ResearchGate +3 et dominait les qualifications solo — souvent le plus rapide en tour isolé. ResearchGate

Ce qui n'a pas fonctionné pour le temps au tour pur

Les contrôleurs **purement neuronaux** (NEAT de Cardamone, WCCI-2008 : victoire initiale SlideShare mais instabilité) SlideShare SlideShare et l'**apprentissage par imitation** (Muñoz et al. : ~20 % plus lent que l'humain) Academia.edu +2 n'ont jamais atteint le niveau des champions. L'**apprentissage par renforcement profond** (DDPG, DQN), développé principalement après 2016, a démontré la capacité d'apprendre à conduire mais **n'a jamais égalé les temps au tour des meilleurs contrôleurs de compétition**. Le DDPG souffre d'exploration inefficace dans l'espace continu, de minima locaux, et de difficultés avec le freinage Slideshare — l'innovation du « frein stochastique » (Ben Lau : ne freiner que 10 % du temps en exploration) a été nécessaire pour tout apprentissage. Yanpanlau Les contrôleurs **purement à règles fixes** (SimpleSoloController de référence, ~100 km/h) étaient non-compétitifs sans optimisation paramétrique. togelius

Approche	Temps au tour	Généralisation	Fiabilité	Coût d'entraînement
Floue + AG (AUTOPIA)	★★★★★	★★★★★	★★★★★	Moyen
Apprentissage piste + CMA-ES (Mr. Racer)	★★★★★	★★★★	★★★★	Moyen
CMA-ES paramétré (COBOSTAR)	★★★★	★★★★	★★★★	Moyen
Neuroévolution (NEAT)	★★★	★★★	★★★	Élevé
RL profond (DDPG)	★★	★★	★★	Très élevé
Règles fixes naïves	★★	★★★	★★★★	Faible

La conclusion centrale est sans appel : les architectures modulaires hybrides, combinant des règles structurées avec une optimisation évolutionnaire des paramètres, surpassent toutes les alternatives pour l'optimisation brute du temps au tour.

L'IBM AI Racing League 2026 : ce que l'on sait

La compétition est officiellement nommée « **IBM AI Racing League – Country Challenge 2026** », et non « IBM Granite Racing ». C'est un défi mondial destiné aux étudiants universitaires. Politechnika Śląska Les équipes construisent un pilote autonome en Python pour TORCS, en utilisant **IBM Granite** comme assistant de développement IA (copilote de code) Medium et en validant des certifications IBM SkillsBuild. Bristol +2 Le circuit obligatoire est **Corkscrew** (reproduction numérique de Laguna Seca dans TORCS), avec un **départ arrêté**. umcs UMCS

La « Règle des 8 » et le cadre de compétition

Le document-cadre officiel (hébergé à l'UMCS en Pologne) impose huit règles : certification IBM SkillsBuild obligatoire (dont Granite), départ arrêté sur le circuit Corkscrew, interdiction de modifier la dynamique interne de TORCS ou le modèle de voiture (seul le code Python IA peut être modifié), vidéos d'équipe avec nom et université, équipes de 5 à 10 membres maximum, temps au tour utilisé comme qualificatif et vidéo d'équipe pour départager les finalistes. umcs UMCS Les soumissions comprennent la vidéo de course, la vidéo d'équipe, un dépôt GitHub et un article de blog décrivant l'approche et l'utilisation d'IBM Granite. UMCS

Les prix incluent un **mentor carrière IBM pendant 3 mois** pour le top 10 régional, UMCS une visite de site IBM et des trophées pour le top 3, et une progression vers l'étape géographique pour le premier. umcs

Les équipes documentées et leurs résultats

L'équipe « **The MonDragons** » (Queen Mary's University of London) a publié un article détaillé sur Medium. Medium Leur approche : contrôleur à règles itérativement optimisé, partant du framework `torcs_jm_par.py` fourni. Medium Ils ont utilisé IBM Granite pour comprendre les capteurs, la physique, et optimiser les paramètres. Medium Leur progression de temps : de **02:33:24 initialement** à **01:47.84** — le meilleur temps publiquement documenté. Un défi majeur a été le changement de modèle de voiture (vers une F1) en milieu de compétition, nécessitant une réécriture complète. Medium

L'**Université de East London** (Sabaad et Ismaeel) a documenté une approche similaire — réglage méthodique et incrémental de la direction, du freinage et de l'accélérateur. Medium Medium Aucun temps spécifique rapporté. D'autres universités participantes confirmées incluent Bristol, la Silesian University of Technology, l'UMCS (Pologne) et l'Universidad Politécnica de Madrid.

Aucun classement public officiel n'a été trouvé. La compétition semble toujours en cours en avril 2026. Aucun résultat final n'a été annoncé. **Toutes les équipes documentées utilisent des approches à règles** Medium — aucune équipe n'a publié d'approche par apprentissage par renforcement. Medium

---

Les temps au tour connus sur Michigan et Corkscrew

Aucun temps au tour individuel en secondes n'est publiquement disponible pour Michigan ou Corkscrew dans le contexte SCR historique. Les compétitions SCR mesuraient la performance en distance parcourue sur un nombre fixe de ticks, ou en points de course F1 — pas en temps au tour individuels. (Togelius) Voici néanmoins les données les plus pertinentes récupérées.

Michigan Oval : COBOSTAR dominait

Lors de la manche **CEC-2009**, les courses se déroulaient sur Michigan, Alpine 2 et Corkscrew. Sur Michigan, **COBOSTAR a obtenu 12 points** (maximum), devançant Onieva & Pelta (8 points) et le Champion CIG-2008 (5 points). (yumpu) (Yumpu) Michigan est un ovale en D avec des virages doux et du banking — un circuit qui teste principalement la **vitesse de pointe** et le maintien de trajectoire en courbe à haute vitesse. Sur des ovales comparables (D-Speedway), les benchmarks FSMDriver donnent environ **60,7 secondes par tour** sur 10 tours. (sbgames) La voiture car1-trb1 peut dépasser **300 km/h** sur les lignes droites. (GitHub)

Corkscrew : Mr. Racer était le plus rapide

Sur Corkscrew au CEC-2009, **Mr. Racer a dominé avec 10 points** (maximum), loin devant COBOSTAR (9) et les autres. (yumpu) (Yumpu) Le Corkscrew est un circuit routier complexe avec des virages serrés, des changements d'élévation et le célèbre « tire-bouchon » de Laguna Seca — très peu de marge d'erreur. Pour l'IBM AI Racing League, le meilleur temps publiquement documenté est **01:47.84** (MonDragons), mais il est vraisemblable que des équipes non-documentées fassent mieux.

Le contrôleur Python le plus rapide : SnakeOil

SnakeOil de Chris X Edwards est le contrôleur Python le plus performant en compétition SCR (GitHub) (Slideshare) — **3e place au SCR 2015**. (Slideshare +2) Sa vitesse de pointe observée est d'environ **276 km/h** (contre 290-293 km/h pour AUTOPIA). (Xed) Edwards souligne que l'**optimisation génétique des paramètres sur un cluster Linux** est la clé de la performance (Xed) — le code de base avec paramètres par défaut est intentionnellement conservateur. (Xed) Le projet est disponible à `xed.ch/project/snakeoil/`. (xed) (GitHub) Autres dépôts notables : `ugo-nama-kun/gym_torcs` (wrapper OpenAI Gym), (GitHub) `yanpanlau/DDPG-Keras-Torcs`, `Alberto-SM/torcs` (Python 3.7), (GitHub) et `bruno147/fsmdriver` (C++, 4e place SCR 2015).

Les paramètres critiques et leurs valeurs optimales

La direction : le paramètre le plus dangereux à haute vitesse

Le paramètre **steerLock = 0,366519 rad (~21°)** est la butée physique de direction. (GitHub) La sortie [-1, +1] est mappée sur cet angle. (GitHub) Le contrôle de direction efficace combine deux composantes : la correction d'angle vers la piste (`S['angle']`) et la correction de position

(`S['trackPos']`). La correction de position par défaut de SnakeOil (`-trackPos × 0,10`) est **trop faible** ; le SimpleDriver utilise `× 0,50`.

L'innovation critique est l'**atténuation de la direction en fonction de la vitesse**. Le SimpleDriver utilise un seuil de sensibilité de **80 km/h** : au-delà, la direction est divisée par `steerLock × (vitesse - 80) × 1,0`. GitHub Sans cette atténuation, toute tentative d'accélérer au-delà de 150 km/h provoque des tête-à-queue. **C'est la modification la plus impactante** pour quiconque augmente la vitesse cible sans adapter la direction.

Le freinage : trois stratégies de référence

La stratégie la plus efficace est le **contrôle sigmoïde** utilisé par le SimpleDriver et AUTOPIA :

```
accel_brake = 2.0 / (1.0 + exp(vitesse - vitesse_cible)) - 1.0
```

GitHub

Cette formule produit un freinage proportionnel et progressif — pas les ajustements incrémentaux de ±0,01 de SnakeOil par défaut, qui sont **désastreusement lents** en réponse.

Le **freinage anticipé** (look-ahead) utilise les capteurs à ±5° (indices 8 et 10) par rapport au centre (indice 9) pour estimer la courbure à venir. La formule du SimpleDriver calcule un angle de virage à partir de la géométrie triangulaire des trois capteurs, puis adapte la vitesse cible proportionnellement. Le seuil **maxSpeedDist = 70 m** définit la distance au-delà de laquelle la piste est considérée « droite ». GitHub Pour les ovales, cette valeur devrait être **150 m ou plus**.

L'**ABS logiciel** est simple mais crucial : si le glissement (vitesse réelle − vitesse moyenne des roues) dépasse **2,0 m/s**, le freinage est réduit proportionnellement. Sans ABS, un freinage violent bloque les roues et provoque une perte de contrôle. Les constantes de référence : `absSlip = 2,0`, `absRange = 3,0`, `absMinSpeed = 3,0 m/s`. GitHub

Les rapports de vitesse : le levier d'amélioration le plus sous-estimé

Le SnakeOil par défaut n'utilise que **3 vitesses** avec des seuils basés sur la vitesse (50 km/h, 80 km/h). xed C'est **le plus gros limiteur de performance** : le véhicule ne dépasse jamais la 3e vitesse, plafonnant à ~100 km/h alors que la car1-trb1 peut atteindre 300+ km/h.

La stratégie optimale utilise des **seuils basés sur le RPM, pas la vitesse**. COBOSTAR (vainqueur CEC-2009) ResearchGate utilise :

- **Passage supérieur : toujours à 9 500 RPM** (toutes les vitesses)
- **Rétrogradage** : 2→1 à 3 300 RPM, 3→2 à 6 200, 4→3 à 7 000, 5→4 à 7 300, 6→5 à 7 700 ResearchGate

Le SimpleDriver est plus conservateur : montée entre 5 000 et 7 000 RPM, descente entre 2 500 et 3 500 RPM. GitHub github La recherche (Becheru et al.) démontre que l'**optimisation des 12**

seuils de vitesse (6 montées + 6 descentes) peut améliorer le temps au tour de **30 %** à elle seule. ResearchGate Academia.edu

Valeurs spécifiques pour un ovale type Michigan

Sur un ovale, la stratégie diffère radicalement des circuits routiers. La vitesse cible sur les lignes droites doit être de **300+ km/h**, et de **200-280 km/h** dans les virages selon le banking. Le freinage doit être **minimal ou absent** si les virages sont suffisamment inclinés. La voiture doit atteindre la **5e ou 6e vitesse** pour l'essentiel du tour. La position de piste optimale n'est pas le centre mais le bord intérieur des virages (la trajectoire de course), ce qui rend la correction `trackPos × 0,5` vers le centre **contre-productive sur ovale**.

Les cinq modifications à plus fort impact par ordre de priorité

Pour un contrôleur basé sur `torcs_jm_par.py` / snakeoil3, voici les changements classés par impact décroissant sur le temps au tour, avec le code de référence correspondant.

Priorité 1 : Passer aux 6 vitesses par RPM. Remplacer les seuils de vitesse par des seuils RPM agressifs : montée à 8 000–9 500 RPM, descente entre 3 000 et 7 000 RPM selon le rapport. Cette seule modification peut réduire le temps au tour de **30 %** en exploitant la pleine bande de puissance du moteur car1-trb1. Academia.edu

Priorité 2 : Vitesse cible dynamique basée sur les capteurs. Remplacer `target_speed = 100` par une formule utilisant le capteur frontal : si la distance frontale dépasse 150 m, viser 300 km/h ; entre 70 et 150 m, viser 200 km/h ; en dessous, adapter proportionnellement (`max(50, distance × 2)`). Sur un ovale, ces seuils doivent être encore plus agressifs.

Priorité 3 : Contrôle sigmoïde accélération/freinage. Remplacer l'ajustement incrémental ±0,01 par `2/(1 + exp(vitesse - cible)) - 1`. Le résultat positif donne l'accélération, le négatif donne le freinage. La réponse est immédiate et proportionnelle au lieu d'accumuler lentement des incréments.

Priorité 4 : Atténuation de la direction en vitesse. Au-delà de 80 km/h, diviser l'angle de direction par un facteur proportionnel à la vitesse excédentaire. Sans cela, augmenter la vitesse cible au-delà de 150 km/h provoque systématiquement des tête-à-queue et des sorties de piste.

Priorité 5 : ABS logiciel et détection anticipée des virages. Ajouter la détection de glissement roues/vitesse et réduire le freinage en cas de blocage. Combiner avec l'utilisation des capteurs à ±5° pour détecter les virages avant d'y entrer, permettant un freinage précoce mais moins violent.

Les erreurs classiques à éliminer absolument

- **N'utiliser que 3 vitesses** : plafonne la vitesse à ~100 km/h, gaspillant les deux tiers du potentiel du véhicule

- **Vitesse cible fixe et basse** : `target_speed = 100` est approprié pour un test de fonctionnement, pas pour une course
- **Ajustement incrémental de l'accélérateur** (±0,01 par tick) GitHub : temps de réponse catastrophique face aux virages rapprochés
- **Direction sans atténuation en vitesse** : provoque des oscillations violentes et des tête-à-queue à haute vitesse
- **Biais de centrage sur ovale** : forcer la voiture au centre de la piste au lieu de suivre la trajectoire de course intérieure
- **Pas de récupération en cas de sortie de piste** : quand `trackPos > 1` ou `< -1`, les capteurs de distance deviennent non fiables (-1) ; un mode de récupération dédié est nécessaire togelius arxiv
- **Ignorer l'embrayage au départ** : sans gestion de l'embrayage, les roues patinent au lancement, coûtant plusieurs secondes
- **Paramètres identiques pour tout type de circuit** : un contrôleur optimisé pour circuit routier freine trop agressivement sur un ovale, et inversement ScienceDirect

---

Synthèse stratégique et perspectives pour le Corkscrew

Le circuit Corkscrew (Laguna Seca) impose des contraintes spécifiques qui le distinguent des ovales et des autres circuits routiers TORCS. Les changements d'élévation brutaux — notamment le célèbre passage du tire-bouchon avec sa descente aveugle — exigent un **freinage anticipé très précis** combiné à une **gestion fine de la direction dans les transitions de pente**. Lapmeta Medium Mr. Racer dominait ce circuit en 2009 yumpu grâce à sa reconstruction géométrique de la piste pendant l'échauffement, ce qui lui permettait de planifier ses trajectoires plusieurs dizaines de mètres à l'avance. Springer

Pour un contrôleur à règles sans phase d'apprentissage, les stratégies les plus pertinentes sur le Corkscrew sont : **utiliser les 19 capteurs de distance pour construire un profil de virage en temps réel** togelius Diva-portal (pas seulement les 3 capteurs centraux), **moduler la vitesse cible par la distance du capteur le plus court** (qui indique le virage le plus proche), et **implémenter une mémoire des positions de dégâts** (à la manière d'AUTOPIA qui multiplie par 0,95 les vitesses cibles dans une zone de 250 m avant chaque point de crash précédent). Slideshare

Le meilleur temps documenté de **01:47.84** (MonDragons) laisse une marge d'amélioration considérable. En implémentant les cinq priorités décrites — particulièrement les vitesses à 6 rapports par RPM et la vitesse cible dynamique — et en ajoutant une logique de freinage anticipé sophistiquée basée sur la géométrie à trois capteurs, un temps bien inférieur à **01:30** devrait être atteignable. L'histoire des compétitions SCR démontre que la combinaison d'une architecture modulaire intelligente avec une optimisation systématique des paramètres — même manuelle et itérative — surpasse toujours les approches monolithiques ou les paramètres fixes. Xed La clé

n'est pas la complexité algorithmique, mais la **précision du réglage paramétrique** sur le circuit cible.